

# Guión de Prácticas 1

## Modularización y Compilación Separada

Metodología de la Programación  
Curso 2025/2026

### Contenidos

|                                                           |   |
|-----------------------------------------------------------|---|
| 1. Introducción al guión                                  | 1 |
| 2. Proyecto de Ejemplo: circulomedio                      | 1 |
| 3. Implementación en módulos separados                    | 2 |
| 4. Implementación en carpetas separadas                   | 4 |
| 5. Automatización de la compilación con ficheros Makefile | 4 |
| 6. Ejercicio a entregar                                   | 7 |

### 1. Introducción al guión

Para esta sesión de prácticas, el alumno deberá entender los conceptos relacionados con la compilación separada (ver tema 12 del libro Garrido, A. “Fundamentos de programación en C++”, Delta Pub., 2005).

### 2. Proyecto de Ejemplo: circulomedio

El objetivo es la creación del tipo de dato **Punto**, como una estructura con dos campos (las coordenadas de un punto 2D), y la del tipo de dato **Circulo**, como una estructura con dos campos (un punto correspondiente al centro y un valor indicando el radio).

Para centrarnos en la compilación separada, se proporciona el código fuente completo que implementa un programa que lee dos círculos desde la entrada estándar y escribe en la salida estándar el círculo que tiene como centro

el punto medio de los dos centros de entrada, y cuyo radio es la mitad de la distancia entre ellos.

Descargue este programa (archivo `circulomedio.zip`) en una carpeta llamada **Version1** y descomprímalo.

A continuación, compile el programa:

```
g++ circulomedio.cpp -o circulomedio
```

Ejecute el programa para comprobar su funcionamiento:

```
./circulomedio
```

También se proporciona un fichero de validación `circulomedio.dat`, que puede usarse como entrada:

```
./circulomedio < circulomedio.dat
```

### 3. Implementación en módulos separados

Cree una nueva carpeta llamada **Version2** donde se hará una nueva versión del programa. A continuación, deberá dividir el programa que ya ha implementado y ejecutado en la sección anterior en los siguientes módulos distintos:

- **Módulo Punto:** Definido en `punto.h` y `punto.cpp`. Contiene el código para manejar la estructura `Punto`.
- **Módulo Circulo:** Definido en `circulo.h` y `circulo.cpp`. Contiene el código para manejar la estructura `Circulo` y hace uso del módulo `Punto`.
- **Módulo Central:** Definido en `central.cpp`. Contiene el código que implementa el programa principal que realiza los cálculos sobre los círculos.

Para evitar dobles inclusiones en los archivos de cabecera, utilice la siguiente estructura:

```
#ifndef _FICHERO_H_
#define _FICHERO_H_
```

```
// Contenido del archivo de cabecera  
  
#endif
```

En los ficheros .h se incluirán la definición de las estructuras y los prototipos de las funciones(su cabecera más punto y coma). Por ejemplo, el contenido de punto.h sería algo así:

```
#ifndef _PUNTO_H  
#define _PUNTO_H  
class Punto {  
private:  
    double x;  
    double y;  
public:  
    Punto(){x=0;y=0;}  
    Punto(double x, double y){this->x=x;this->y=y;}  
    ...  
};  
double distancia(Punto p1, Punto p2);  
Punto puntoMedio(Punto p1, Punto p2);
```

Para poder compilar bien los ficheros cpp deberán incluirse donde sea necesario, los ficheros .h con la directiva:

```
#include "punto.h"
```

Cuando tenga los tres módulos, deberán compilarse para obtener los archivos objeto:

```
g++ -c punto.cpp -o punto.o  
g++ -c circulo.cpp -o circulo.o  
g++ -c central.cpp -o central.o
```

Luego, se deben enlazar para obtener el ejecutable final:

```
g++ punto.o circulo.o central.o -o circulomedio
```

## 4. Implementación en carpetas separadas

Cree una nueva carpeta llamada **Version3** donde se hará una nueva versión del programa. Dentro de ella, deberá crear los directorios **include** para los ficheros **.h**, **src** para los ficheros **.cpp**, **obj** para los ficheros **.o**, **lib** para alojar las bibliotecas y **bin** para los ejecutables. Se crearán también las carpetas **doc** para documentación y **data** para ficheros auxiliares y se colocará cada fichero en su sitio.

Una vez ordenados los ficheros por carpetas, se procede a compilar, cogiendo cada fichero desde su carpeta y colocando cada fichero de salida en su carpeta correspondiente.

```
g++ -c src/punto.cpp -o obj/punto.o -Iinclude
g++ -c src/circulo.cpp -o obj/circulo.o -Iinclude
g++ -c src/central.cpp -o obj/central.o -Iinclude
g++ obj/central.o obj/punto.o obj/circulo.o -o bin/circulomedio
```

## 5. Automatización de la compilación con ficheros Makefile

Gracias al modelo de dependencias entre los ficheros de nuestro proyecto tras hacer modificaciones en un módulo no necesitamos reconstruir el proyecto completo. Cuando modificamos algún fichero es necesario reconstruir uno o varios ficheros que dependen de él, lo que hace que a su vez sea necesario reconstruir los ficheros que dependan de los nuevos ficheros reconstruidos, y así sucesivamente. El problema es que, dependiendo del módulo al que afecten las modificaciones, lo que tenemos que reconstruir podrá variar. Por ejemplo, si modificamos **circulo.cpp** debemos reconstruir **circulo.o**, y el ejecutable **circulomedio**. Para evitar acordarnos de qué partes se han de reconstruir en función de cuales se han modificado, disponemos de herramientas que automatizan todo el proceso. En nuestro caso usaremos la utilidad **make**. Su cometido es leer e interpretar un fichero en el que se almacenan una serie de reglas que reflejan el esquema anterior de dependencias. Lo habitual es que a este fichero de reglas se le llame Makefile. El aspecto de una regla es el siguiente:

|                                              |
|----------------------------------------------|
| Objetivo : Lista de dependencias<br>Acciones |
|----------------------------------------------|

En donde:

- **Objetivo:** Esto es lo que queremos construir. Habitualmente es el nombre de un fichero que se obtendrá como resultado del procesamiento de otros ficheros. Más adelante veremos que no tiene que ser, necesariamente, un nombre de fichero.
- **Lista de dependencias:** Esto es una lista de items de los que depende la construcción del objetivo de la regla. Normalmente los items son ficheros o nombres de otros objetivos. Al dar esta lista de dependencias, la utilidad make debe asegurarse de que han sido todas satisfechas antes de poder alcanzar el objetivo de la regla.
- **Acciones:** Este es el conjunto de acciones que se deben llevar a cabo para conseguir el objetivo. Normalmente serán instrucciones como las que hemos visto antes para compilar, enlazar, etc.

**IMPORTANTE:** Observa que al escribir estas reglas en el fichero Makefile, las acciones se deben sangrar (o tabular) con un carácter tabulador.

Por ejemplo, la siguiente sería una regla válida:

```

circulomedio : obj/central.o obj/punto.o obj/circulo.o
    g++ -o bin/circulomedio obj/central.o obj/punto.o obj/circulo.o

```

donde el objetivo es la construcción del ejecutable `bin/circulomedio`. En la lista de dependencias hemos puesto la lista de ficheros de los que depende, o lo que es lo mismo, estamos diciendo que para poder construir `bin/circulomedio`, previamente han de haber sido contruidos adecuadamente los ficheros `obj/central.o`, `obj/punto.o` y `obj/circulo.o`. Finalmente la acción que hay que llevar acabo para generar el objetivo es la ejecución de `g++` con los parámetros que vemos en el ejemplo. Si al aplicar la regla, se detecta que alguno de los items de la lista de dependencias no existe o necesita ser actualizado, entonces pasará a realizar esa actualización, para crearlo de nuevo, antes de aplicar las acciones de la regla actual.

El fichero Makefile completo para la compilación de la version modularizada del proyecto sería el siguiente:

```
circulomedio: obj/central.o obj/circulo.o obj/punto.o
    g++ -o bin/circulomedio obj/central.o obj/circulo.o obj/punto.o
obj/central.o: src/central.cpp include/punto.h include/circulo.h
    g++ -Iinclude -c src/central.cpp -o obj/central.o
obj/punto.o: src/punto.cpp include/punto.h
    g++ -Iinclude -c src/punto.cpp -o obj/punto.o
obj/circulo.o: src/circulo.cpp include/circulo.h
    g++ -Iinclude -c src/circulo.cpp -o obj/circulo.o
clean:
    rm -f bin/* obj/*.o
```

Se ha añadido una regla con nombre `clean` que borra los ficheros `.o` y el ejecutable. Una vez construido el fichero `Makefile`, ejecutando `make` sin parámetros se aplicará la primera regla del `Makefile`. Para ejecutar una regla específica:

```
make circulomedio
make clean
```

La construcción del fichero `Makefile` se puede generalizar de la siguiente forma para la estructura de proyecto que hemos descrito:

```

CXX = g++
SRC_DIR = src
OBJ_DIR = obj
INC_DIR = include
BIN_DIR = bin
TARGET = circulomedio

# Obtener todos los archivos fuente en src/
SOURCES = $(wildcard $(SRC_DIR)/*.cpp)

# Convertir los archivos fuente en archivos objeto en obj/
OBJECTS = $(SOURCES:$(SRC_DIR)/%.cpp=$(OBJ_DIR)/%.o)

DEPS = $(SOURCES:$(SRC_DIR)/%.cpp=$(OBJ_DIR)/%.d)

# Regla principal para compilar el ejecutable
$(TARGET): $(OBJECTS)
    $(CXX) $(OBJECTS) -o $(BIN_DIR)/$(TARGET)

# Regla para compilar cada archivo fuente en un archivo objeto
$(OBJ_DIR)/%.o: $(SRC_DIR)/%.cpp $(OBJ_DIR)/%.d
    $(CXX) -c $< -o $@ -I$(INC_DIR)

# Generar las dependencias para cada archivo .cpp
$(OBJ_DIR)/%.d: $(SRC_DIR)/%.cpp
    $(CXX) -MM $< -MF $@ -MT $(OBJ_DIR)/$*.o -I$(INC_DIR)

-include $(DEPS)

clean:
    rm -f $(OBJ_DIR)/*.o $(OBJ_DIR)/*.d $(BIN_DIR)/$(TARGET)

```

## 6. Ejercicio a entregar

Se proporciona un fichero `cancion.h` con la definición de la clase `Cancion` para almacenar distinta información acerca de canciones como puede ser el título, el artista, el año de lanzamiento, etc. Se pide lo siguiente:

1. Implemente en un fichero `cancion.cpp` cada uno de los métodos descritos en la definición de la clase tal como se establece en el fichero `cancion.h`.

2. Implemente la función `filtrarCancionesPorAnio` en el fichero `cancion.cpp`.
3. En el fichero `main.cpp` implemente el programa principal que debe realizar las siguientes tareas:
  - a) Leer por la entrada estándar los datos de una canción separados por comas.
  - b) Crear un vector de canciones, se leerá con antelación el número de canciones que se espera recibir por la entrada estándar.
  - c) Leer un año y filtrar las canciones de ese año llamando a la función `filtrarCancionesPorAnio`.
  - d) Mostrar por pantalla los datos de las canciones lanzadas en ese año **recorriendo el vector ya filtrado**. Añadir un método `toString()` a la clase `Cancion` para mostrar todos los atributos de cada canción de la forma `<nombre de atributo>: <valor del atributo>`.
4. Entregar el ejercicio en un fichero `.zip` con una estructura de carpetas como la que se ha visto en el ejemplo descrito en este guión y su correspondiente fichero `Makefile`.